

Doubly Linked List – Traverse In Reverse And Time Complexity

Program :

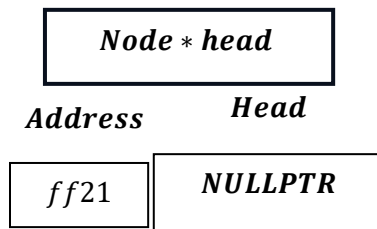
```
void traverseInReverse()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    { // Go to the last node
        temp = temp->next;
    }

    cout << "List (Reverse): NULL <- ";
    while (temp != nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->prev;
    }
    cout << "NULL" << endl;
}
...
case 11:
    traverseInReverse();
    break;
```

Program Analysis

A. When LIST is Empty

```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return ;
}
```

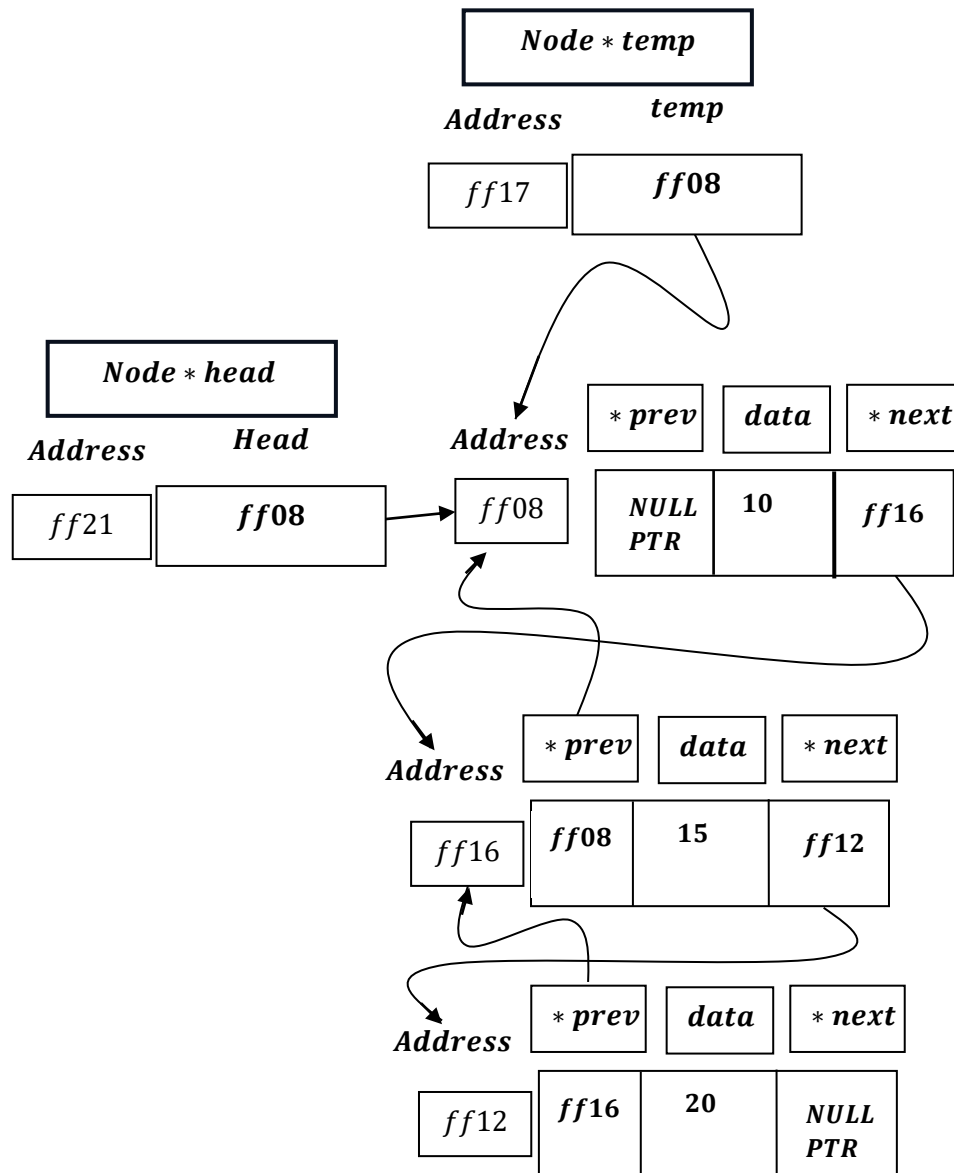


Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

B. When LIST is not Empty

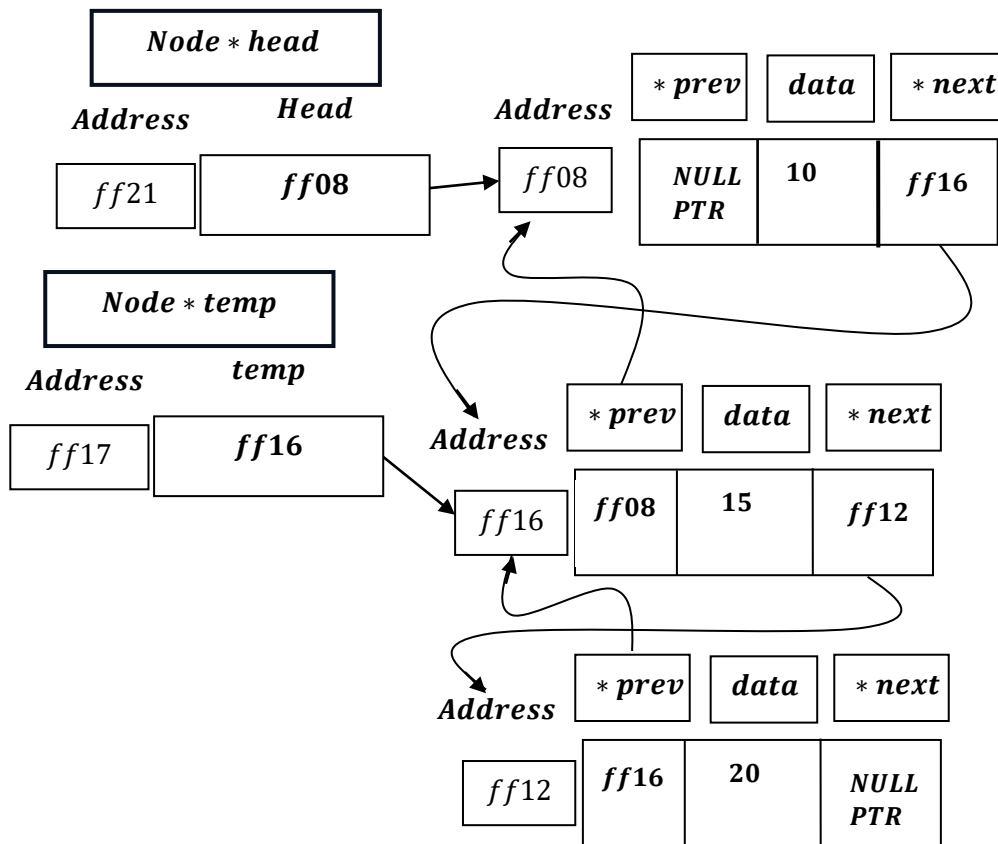
```
Node *temp = head;
```



```
while (temp->next != nullptr)
{ // Go to the last node
    temp = temp->next;
}
```

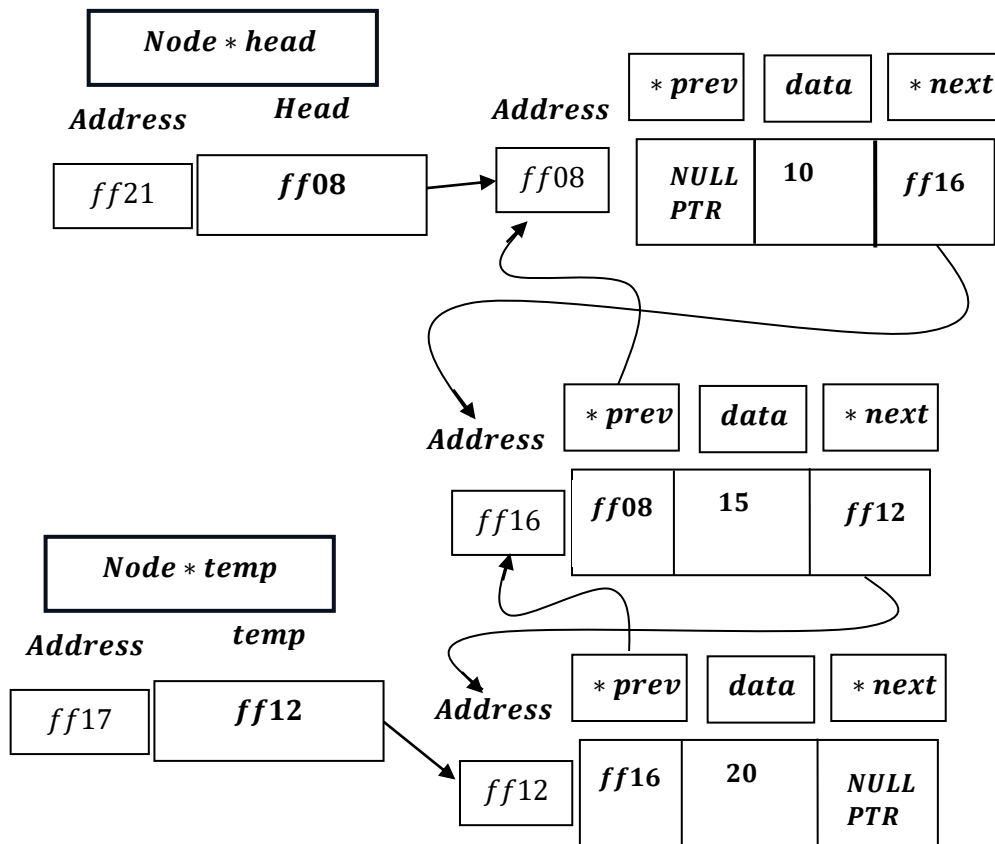
Temp $\neq$ *NULLPTR* is true , hence:

temp = *temp* $\rightarrow$ *next* = *ff16* .



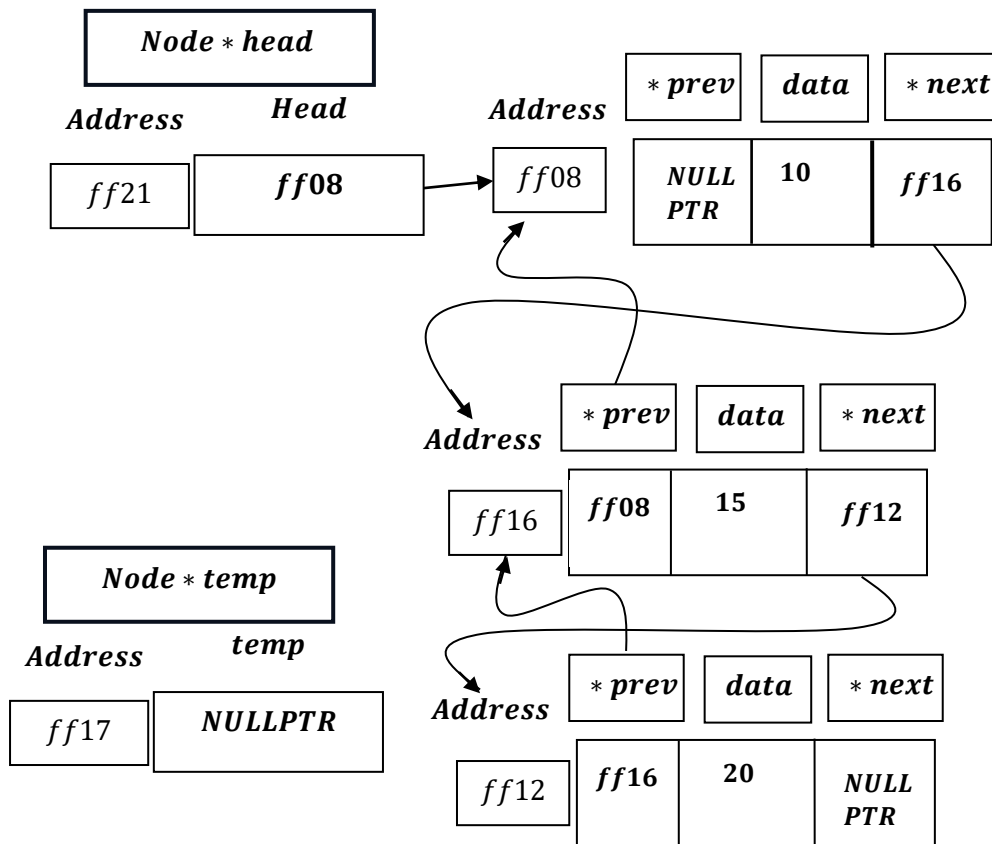
Temp $\neq$ *NULLPTR* is true , hence:

temp = *temp* $\rightarrow$ *next* = *ff12* .



Temp $\neq$ *NULLPTR* is true , hence:

temp = *temp* $\rightarrow$ *next* = *NULLPTR* .



Now, TEMP = NULLPTR, hence while (TEMP ≠ NULLPTR) → condition fails .

```

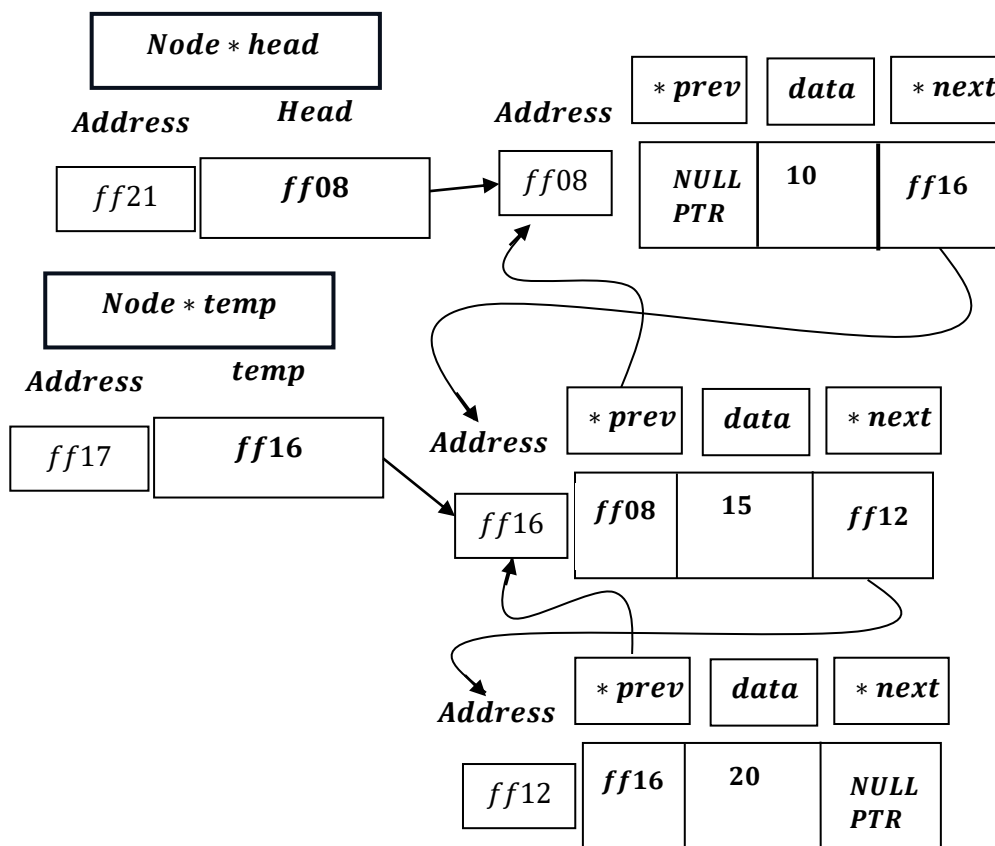
cout << "List (Reverse): NULL <- ";
while (temp != nullptr)
{
    cout << temp->data << " <-> ";
    temp = temp->prev;
}
cout << "NULL" << endl;

```

Temp ≠ NULLPTR is true , hence:

cout << *temp* → *data* << <->; ⇒ Output: NULL < -20 < ->

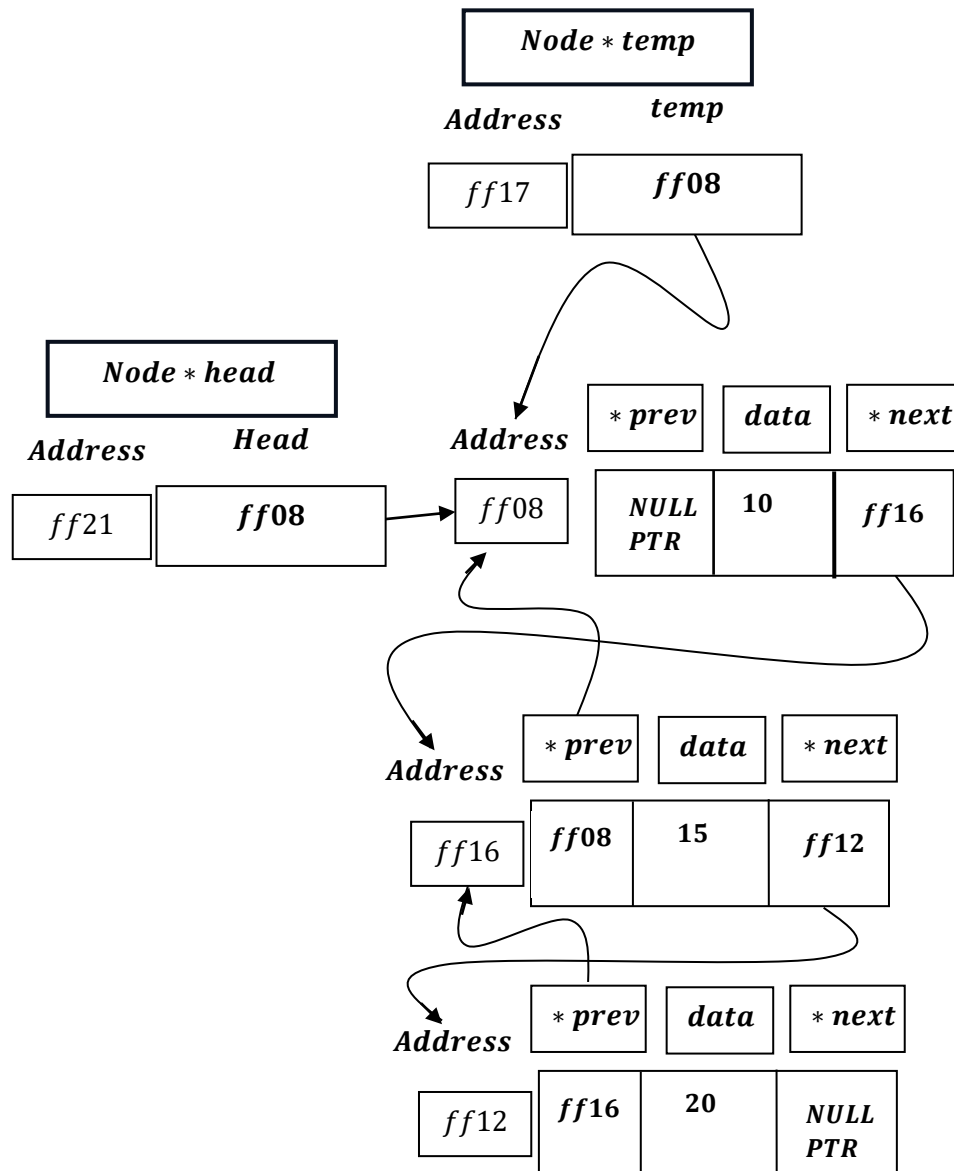
temp = *temp* → *prev* = *ff16* .



Temp ≠ NULLPTR is true , hence:

cout << *temp* → *data* << <->; ⇒ Output: NULL < -20 < -> 15 < ->

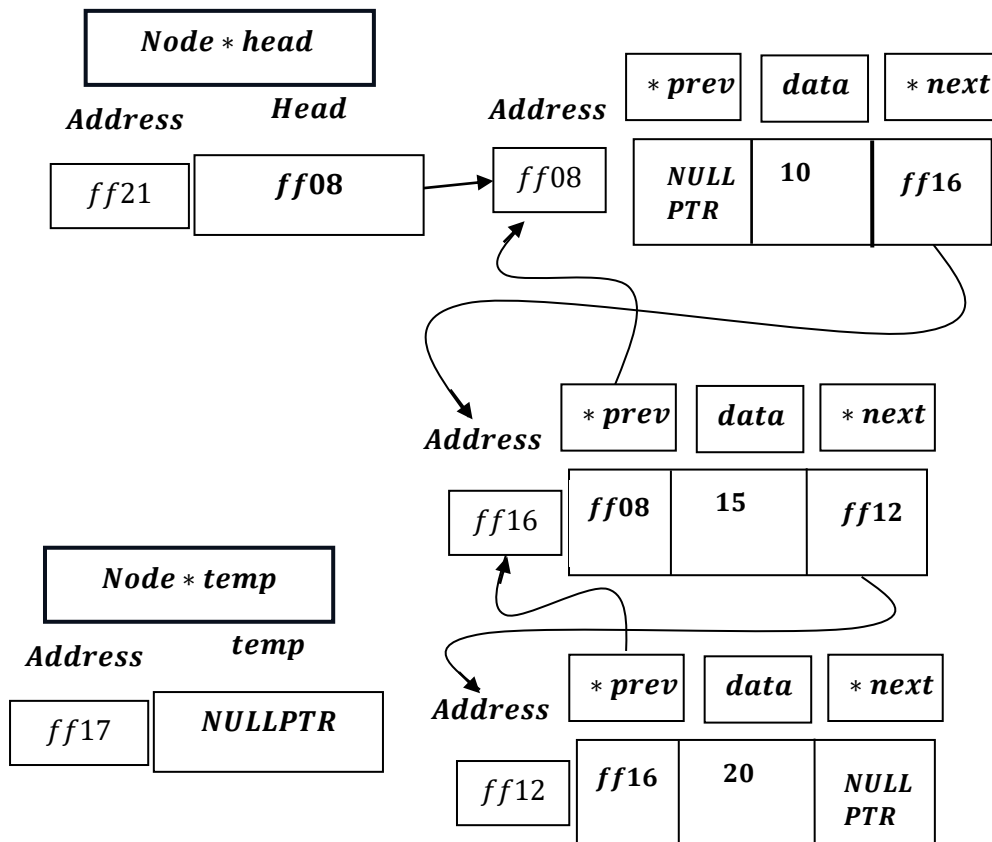
temp = *temp* → *prev* = *ff08* .



Temp $\neq$ NULLPTR is true , hence:

`cout << temp -> data << <->;` $\Rightarrow$ Output: NULL < -20 < -> 15 < -> 10 < ->

`temp = temp -> prev = NULLPTR.`

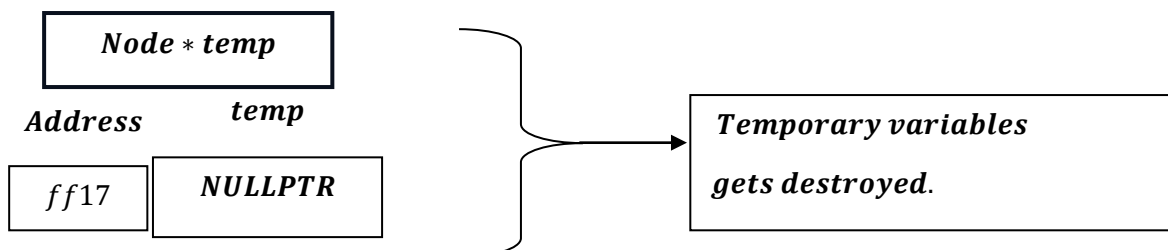


Now, $TEMP = NULLPTR$, hence while ($TEMP \neq NULLPTR$) $\rightarrow$ condition fails .

```
cout << "NULL" << endl;
```

$cout << "NULL" << <->; \Rightarrow$ Output: $NULL <-> -20 <-> 15 <-> 10 <-> NULL$

As function ends ,Temp pointer gets destroyed:



Doubly Linked List – Traverse In Reverse [Working Summary]:

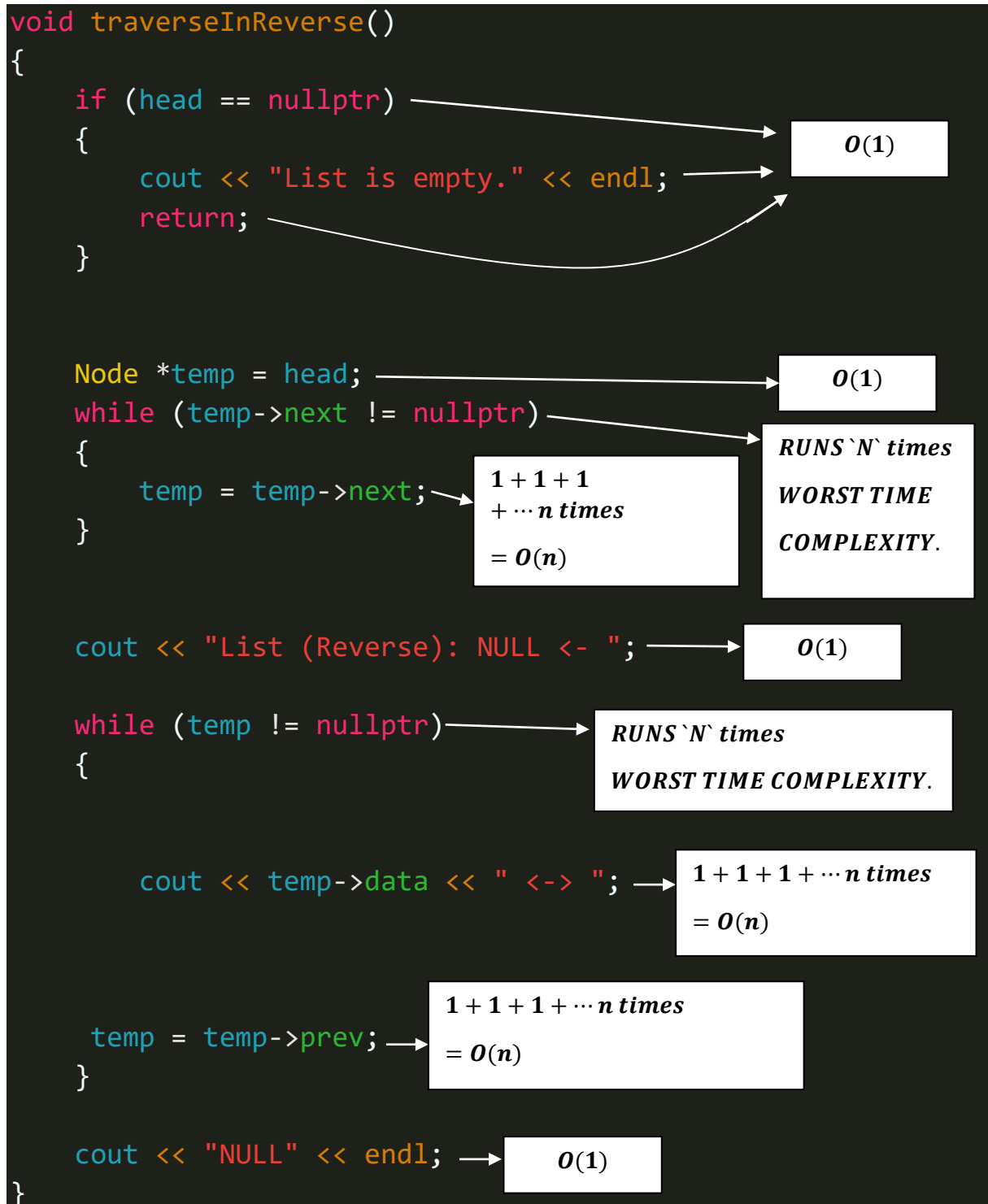
This C++ function, `traverseInReverse`, prints all the elements of a **doubly linked list** in reverse order, from tail to head.

Working Summary

1. **Handles Empty List:** The function first checks if the list is empty (`head == nullptr`). If it is, it prints an "List is empty." message and stops.
2. **Finds the Last Node:** It initializes a temporary pointer `temp` at the `head`. A `while` loop then traverses the list until it reaches the very last node (where `temp->next` is `nullptr`).
3. **Initiates Reverse Print:** After finding the tail, it prints a header `List (Reverse):` `NULL <-` to start the output.
4. **Traverses Backward and Prints:** The function enters a second `while` loop that continues as long as `temp` is not `nullptr`. In each iteration:
 - o It prints the `data` of the current node.
 - o It moves backward to the **previous** node using the `prev` pointer (`temp = temp->prev`). This is the key step that makes reverse traversal possible and indicates the structure is a doubly linked list.
5. **Finishes Output:** Once the loop finishes (after reaching the node before the original `head`), it prints `NULL` to signify the end of the reverse-printed list. For a list with 10, 20, and 30, the output would be:

```
List (Reverse): NULL <- 30 <-> 20 <-> 10 <-> NULL
```

Time Complexity Of Traverse In Reverse Function Module



Program Analysis

Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If head = nullptr, this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes, $O(1)$ time.

Worst Case(When List is not Empty)

→ Node * temp = * head; this assignment takes constant time i. e. $O(1)$.

→ while(temp != nullptr) checking becomes true, then:

→ temp = temp → next; such assignment takes, $1 + 1 + \dots n$ times = n ;

Therefore, total loop time complexity: $O(n)$.

→ cout << "List (Reverse) : NULL < - "; takes constant time to execute i. e. $O(1)$.

→ while(temp != nullptr) checking becomes true, then:

→ cout << temp → data << " < -> ";, this statement gets executed: $1 + 1 + \dots n$ times = n ;

→ temp = temp → prev; such assignment takes, $1 + 1 + \dots n$ times = n ;

Therefore, total loop time complexity: $2n$ or $O(n)$.

→ cout << "NULL " takes constant time to execute i. e. $O(1)$.

Hence total time complexity : $O(1) + O(n) + O(1) + O(n) + O(1) = O(n)$.

Therefore takes $O(n)$ time .

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list.

If the list is empty, the execution time is constant (best case).

If the list is not empty, the execution time is proportional to the length of the list (worst case).

There's no element of randomness or varying input data to consider.

Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here.

Hence , Traverse In Reverse function doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Traverse In Reverse</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case (= Worst Case)</i>	$\Theta(n)$
